Residencias UNAL Sistema de gestión y asignación prioritaria de cupos Universitarios:

Primera Entrega: Fundamentos, Diseño e Implementación Inicial

Laura Victoria Flórez Forero, Juan Sebastian Manrique Mesa, Jhon Alexander Berdugo Pasachoa
Universidad Nacional de Colombia – Sede Bogotá
Bogotá D.C., Colombia
{lflorezf, jumanriqueme, jberdugop}@unal.edu.co

***Resumen*—Este documento presenta el diseño e implementación inicial del sistema para la gestión y asignación de cupos del programa de Residencias Universitarias de la Universidad Nacional de Colombia. El sistema aborda el problema de asignación equitativa de recursos limitados mediante un modelo de prioridad basado en el puntaje socioeconómico de los estudiantes solicitantes (a menor puntaje, mayor prioridad). Para garantizar una eficiencia computacional óptima en el acceso y ordenamiento de la información, se propone un diseño híbrido compuesto por una Tabla Hash (`HashMap`) para el acceso directo por ID en tiempo promedio $O(1)$, y un Montículo Binario (`PriorityQueue`) que mantiene la prioridad en tiempo $O(\log N)$. Se detalla la arquitectura modular desarrollada en Java, la especificación de requerimientos, la justificación analítica de las estructuras de datos elegidas y el estado actual de los componentes construidos.**

***Index Terms*—Asignación de recursos, Tabla Hash, Montículo Binario, Cola de Prioridad, Complejidad Computacional, Java.**

I. Introducción y Planteamiento del Problema

I-A. Contexto General

La Universidad Nacional de Colombia (UNAL) ofrece servicios de bienestar universitario con el fin de promover la permanencia estudiantil y mitigar la deserción por factores socioeconómicos. Uno de los apoyos más críticos es el servicio de residencias universitarias, destinado a brindar alojamiento a estudiantes que demuestren vulnerabilidad económica o residan fuera de la ciudad sede.

I-B. Situación Específica

La demanda de cupos para las residencias excede considerablemente la capacidad instalada disponible. En consecuencia, el proceso de adjudicación debe guiarse por principios estrictos de transparencia, justicia y eficiencia. A cada estudiante solicitante se le asigna un puntaje socioeconómico estandarizado en una escala de 0 a 100. En este esquema, un menor puntaje representa una mayor necesidad socioeconómica y, por ende, una prioridad superior en la asignación del cupo.

I-C. Dificultades Computacionales y Relevancia

El diseño de un sistema informático para este problema involucra dos necesidades operativas que deben resolverse de forma eficiente:

1. **Búsqueda y consulta individual directa:** Las autoridades administrativas requieren consultar, modificar o eliminar la información de un estudiante particular mediante su identificador único (`ID`) de manera instantánea.
2. **Ordenamiento dinámico por prioridad:** El sistema debe mantener constantemente organizadas las solicitudes según el puntaje socioeconómico para proceder a la asignación de cupos en tiempo real, soportando actualizaciones frecuentes de puntajes sin perder el orden relativo.

Un enfoque ingenuo utilizando listas contiguas o no ordenadas requeriría ordenamientos repetidos con costo $O(N \log N)$ o búsquedas lineales $O(N)$, afectando la escalabilidad del sistema. Por ello, este proyecto explora una solución basada en la combinación de estructuras de datos optimizadas.

II. Objetivos

II-A. Objetivo General

Diseñar e implementar un sistema funcional para la gestión y asignación prioritaria de Residencias UNAL, empleando estructuras de datos que garanticen eficiencia en las operaciones de búsqueda por ID y ordenamiento por prioridad socioeconómica.

II-B. Objetivos Específicos

- Modelar el dominio del problema mediante una arquitectura orientada a objetos en lenguaje Java.
- Seleccionar y justificar de forma técnica las estructuras de datos para lograr búsquedas por ID en $O(1)$ e inserciones/extracciones prioritarias en $O(\log N)$.
- Implementar las operaciones principales del sistema: registro, consulta por ID, modificación de puntaje con reordenamiento automático, eliminación de registros y asignación de cupos.
- Analizar la complejidad temporal y espacial de las operaciones soportadas por el sistema.

III. Alcance y Limitaciones

III-A. Alcance de la Entrega Inicial

Esta primera entrega abarca la definición del núcleo del sistema en Java, permitiendo:

- Registrar estudiantes con ID, Nombre Completo y Puntaje socioeconómico (0–100).
- Gestionar la asignación de un número variable de cupos.
- Realizar la distinción entre estudiantes beneficiarios y no beneficiarios.
- Ejecutar pruebas mediante una interfaz por consola de comandos.

III-B. Limitaciones y Supuestos

- **Persistencia:** En esta versión, la información se almacena exclusivamente en memoria principal (*RAM*). La persistencia en archivos o bases de datos se incorporará en entregas posteriores.
- **Interfaz simple:** La interfaz de menú usada para ejecutar pruebas es básica y tiene un diseño mejorable, por lo que esperamos integrar una mejor interfaz en una próxima entrega.
- **Criterio de Desempate:** En caso de empate en el puntaje socioeconómico, se otorga prioridad al estudiante con el menor identificador (`ID`).

IV. Requerimientos Principales

IV-A. Requerimientos Funcionales

- **Registro:** Permitir ingresar un estudiante especificando ID (entero positivo único), nombre completo y puntaje (0 a 100).
- **Consulta** Consultar los datos de un estudiante de forma directa a través de su ID único.
- **Modificación de Puntaje:** Permitir actualizar el puntaje socioeconómico de un estudiante existente, garantizando que el sistema reajuste su posición de prioridad.
- **Eliminación:** Eliminar a un estudiante del sistema mediante su ID.
- **Definición de Cupos:** Configurar el número total de cupos disponibles para el periodo académico.
- **Listado de Solicitudes:** Listar todos los estudiantes ordenados de manera creciente según su puntaje socioeconómico.
- **Proceso de Asignación:** Asignar los cupos disponibles a los estudiantes de menor puntaje y generar dos listas diferenciadas: beneficiarios y no beneficiarios.

IV-B. Requerimientos No Funcionales

- **Eficiencia:** La búsqueda directa debe operar en tiempo constante promedio $O(1)$.
- **Integridad de Datos:** Impedir IDs duplicados o puntajes fuera del rango $[0, 100]$.

V. Trabajo Relacionado Inicial

La asignación de recursos limitados basada en prioridades es un problema habitual en el desarrollo de software. Existen ejemplos similares en sistemas del mundo real, como la asignación de turnos en salas de urgencias médicas, la distribución de becas de estudio o la atención de solicitudes en plataformas de atención al cliente.

En este tipo de sistemas, es común combinar dos estrategias: una para buscar y actualizar la información de una persona de forma inmediata a través de un código o ID único, y otra para mantener a los solicitantes ordenados automáticamente según su nivel de necesidad. Esto permite que el sistema responda de forma rápida ante consultas individuales y, al mismo tiempo, garantice una entrega justa de los recursos disponibles a quienes más lo requieren.

VI. Diseño de la Solución

VI-A. Arquitectura del Sistema

El sistema se estructuró bajo un esquema modular con clara separación de funciones:

- `Estudiante`: Clase constante para el ID, y dinámica controlada para el puntaje. Redefine `equals` y `hashCode` basándose en el ID único.
- `SistemaResidencias`: Clase contenedora de la lógica principal y las estructuras de datos.
- `ResultadoAsignacion`: Registro (`record`) estático que organiza las listas de beneficiarios y no beneficiarios.
- `Main`: Interfaz por consola encargada de la interacción con el usuario y detección de excepciones.

VI-B. Lógica de Prioridad

Para ordenar a los estudiantes se definió un comparador jerárquico expresado formalmente en Java como:

```
Comparator<Estudiante> ORDEN_PRIORIDAD =
    Comparator.comparingInt(Estudiante::
        getPuntajeSocioeconomico)
            .thenComparingInt(Estudiante::getId
                );
```

Este comparador asegura que a menor puntaje mayor prioridad, y ante igualdad de puntaje, el ID menor prevalezca.

VII. Estructuras de Datos Seleccionadas

Para cumplir con las restricciones operativas y de complejidad, se integraron dos estructuras de datos complementarias de la biblioteca estándar de Java (`java.util`):

VII-A. 1. Tabla Hash (*`HashMap<Integer, Estudiante>`*)

- **Rol:** Mapear el ID único (`Integer`) al objeto `Estudiante`.
- **Justificación:** Permite la búsqueda, verificación de existencia e inspección de estudiantes en tiempo constante promedio $O(1)$.
- **Alternativas consideradas:** Un arreglo contiguo o `ArrayList` requeriría $O(N)$ para búsquedas por ID

salvo que se mantenga ordenado, lo cual costaría $O(N)$ por inserción.

VII-B. 2. Cola de Prioridad (`PriorityQueue<Estudiante>`)

- **Rol:** Mantener el conjunto de estudiantes organizados de acuerdo con el comparador `ORDEN_PRIORIDAD`.
- **Justificación:** Facilita la extracción del estudiante con mayor necesidad en tiempo $O(\log N)$ e inserción en tiempo $O(\log N)$.
- **Alternativas consideradas:** Mantener una lista global y ordenarla con *QuickSort* o *TimSort* en cada consulta costaría $O(N \log N)$ por invocación.

VII-C. Análisis de Complejidad de las Operaciones

Cuadro I
Análisis de Complejidad Temporal y Espacial

Operación	Temporal (Peor/Promed.)	Espacial
`registrarEstudiante`	$O(\log N)$	$O(1)$
`buscarPorId`	$O(1)$	$O(1)$
`modificarPuntaje`	$O(N)$	$O(1)$
`eliminarEstudiante`	$O(N)$	$O(1)$
`listarPorPuntaje`	$O(N \log N)$	$O(N)$
`asignarCupos`	$O(N \log N)$	$O(N)$

En la implementación actual, la eliminación de un elemento en `PriorityQueue` de Java (`remove(Object)`) realiza una búsqueda lineal en el arreglo interno del montículo, con un costo $O(N)$. La re-inserción posterior toma $O(\log N)$.

VIII. Estado de la Implementación

El proyecto cuenta actualmente con una primera versión completamente funcional y ejecutable que satisface los requerimientos trazados para la Entrega 1.

Cuadro II
Estado de los Componentes del Sistema

Componente	Descripción	Estado
`Estudiante`	Guarda la información y datos básicos de cada estudiante.	Completado
`SistemaResidencias`	Maneja el registro, búsquedas y la asignación de cupos.	Completado
`ResultadoAsignacion`	Guarda las listas finales de beneficiarios y no beneficiarios.	Completado
`Main`	Menú interactivo en consola para usar el sistema.	Completado
Persistencia	Guardado en archivos/Base de datos.	Pendiente
Árbol Indizado	Mejora para actualizar datos de forma más rápida.	Pendiente

IX. Discusión y Trabajo Futuro

La combinación de `HashMap` y `PriorityQueue` ha demostrado ser altamente efectiva para la escala inicial del proyecto, resolviendo con elegancia la dualidad entre consulta por ID y ordenamiento prioritario.

Sin embargo, el análisis técnico permitió identificar una oportunidad clave de mejora: la operación `modificarPuntaje` requiere retirar el objeto del `PriorityQueue` mediante `remove(Object)`, lo cual toma tiempo lineal $O(N)$.

Para las entregas 2 y 3 se planean las siguientes mejoras:

1. **Implementación de árboles** Reemplazar la cola de prioridad estándar por una estructura tipo árbol que mantenga punteros/índices desde la Tabla Hash hacia la posición del elemento en el árbol del montículo. Esto reducirá el costo de actualización y eliminación a $O(\log N)$.
2. **Persistencia de Datos:** Incorporar lectura y escritura en una base de datos para conservar la información entre ejecuciones.
3. **Interfaz Gráfica (GUI):** Desarrollar una interfaz visual amigable para facilitar las labores administrativas del personal de Bienestar UNAL.

X. Conclusiones

Se estableció con éxito la base conceptual, computacional y técnica del sistema de Residencias UNAL. Mediante el uso acoplado de una Tabla Hash y una Cola de Prioridad, se logró resolver de manera justificada el problema de asignación transparente y priorizada de alojamiento universitario. La primera versión implementada en Java verifica la viabilidad operativa del sistema y sirve como cimiento sólido sobre el cual se incorporarán optimizaciones algorítmicas y funcionalidades avanzadas en las entregas futuras.